

实验八 Milestone 2 (M2) 联调 测试与流处理阶段交付

学号： 9109223091

姓名： 骆开丹

日期： 2026.5.1

一、实验名称

Milestone 2 (M2) 联调 测试与流处理阶段交付

二、实验目的

- 配置化编排：能够将散乱的手脚手架脚本统一为一个可由命令行参数（CLI）驱动运行的工程启动入口。
- 死信队列（DLQ）容错处理：学会在处理高速流数据时截获（Try-Except）脏数据，防止异常日志导致整个消费者进程崩溃。
- 混沌容错测试（Chaos Testing）：利用模拟的异常数据、极限速率对系统进行多项压力测试，验证系统的背压机制能否在长时段下保持韧性。
- 高质量项目开源式交付：学会与 AI 协作生成具有强视觉规范（如内置时序图、数据架构图）的项目 README 文档。

三、实验软硬件环境

- 操作系统： Windows 11
- 核心运行时： Python 3.13.2、Node.js 20.12.0（LTS版，用于运行Qwen Code）
- 研发工具： Visual Studio Code（集成终端 + Notebook 插件）、Git
- 核心依赖库： `threading`、`queue`、`csv`、`datetime`、`random`、`time`、`pathlib`、`dataclasses`、`matplotlib`、`pandas`、`scikit-learn`、`joblib`、`numpy`、`json`、`logging`、`argparse`

四、实验步骤与核心操作

任务1：从"散装手架"到"工业级 Pipeline"的整合重构

1.1 CLI 命令行编排实现

基于实验七 `task3_end_to_end_scoring.py` 的 Producer-Consumer 多线程架构进行重构，使用 Python 内置 `argparse` 模块构建统一入口 `run_pipeline.py`。支持的核心参数包括：

参数	说明	默认值
<code>--qps</code>	生产者速率（条/秒）	20
<code>--queue-limit</code>	队列最大容量（0=无限）	200
<code>--duration</code>	实验运行时长（秒）	30
<code>--dataset</code>	输入 CSV 数据集路径	UserBehavior.csv
<code>--model</code>	模型文件路径	model.pkl
<code>--output</code>	打标结果输出 CSV	scored_events.csv
<code>--chaos</code>	启用混沌测试模式	False
<code>--dirty-ratio</code>	异常数据注入比例	0.01

启动命令示例：

```
python run_pipeline.py --qps 20 --queue-limit 200 --duration 30
![[Pasted image 20260502092643.png]]
```

所有参数在启动时通过 logging 模块输出确认，确保配置正确传递至底层管道。

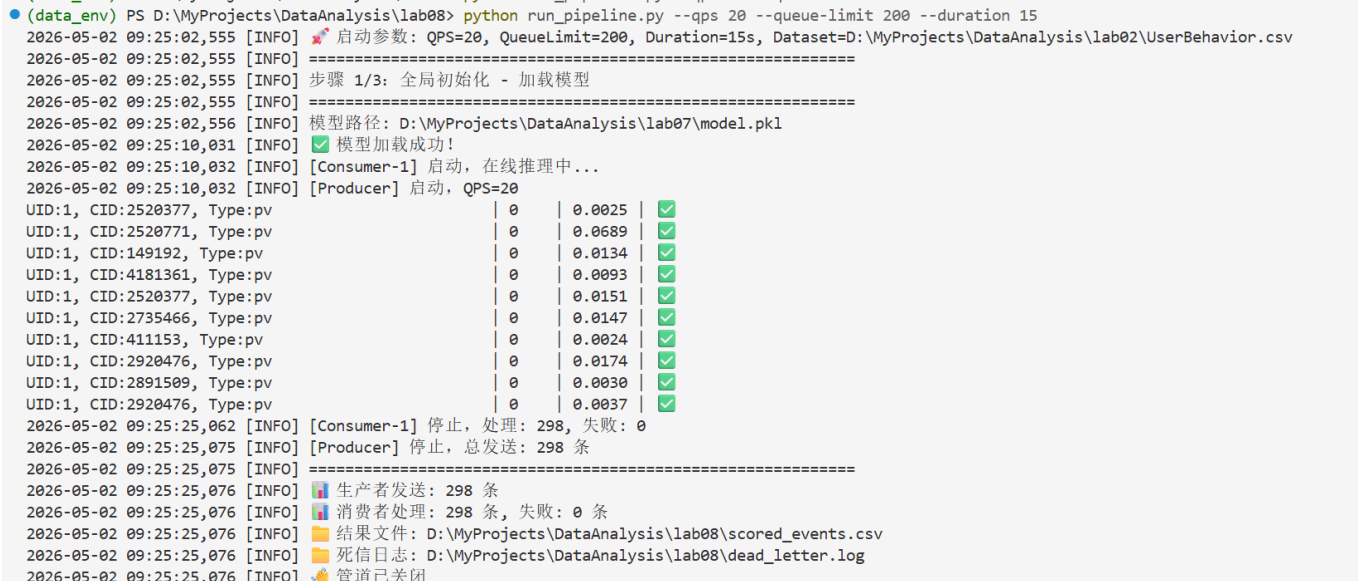
1.2 容错机制与死信降级实现

在 ScoringConsumer.run() 主循环中，对特征提取与模型推理环节包裹强健的 try-except 隔离：

```
# ===== 核心容错区 =====
try:
    features = self._extract_features(event)
    event["predicted_label"] = int(self.model.predict(features)[0])
    event["buy_probability"] = float(self.model.predict_proba(features)[0][1])
except Exception as e:
    self.total_failed += 1
    event["error"] = str(e)[:50]
    write_to_dead_letter(event, str(e))
    # 进程不崩溃，继续处理下一条

# =====
```

异常数据通过 write_to_dead_letter() 函数追加写入 dead_letter.log，每条记录包含时间戳、错误信息及原始数据，格式为 JSON。验证截图如下：



任务2：混沌容错测试（Chaos Testing）

2.1 异常数据

注入实在 ChaosProducer 类中，以 1% 概率随机生成三种异常类型：

- missing_field：随机删除一个字段，模拟埋点数据残缺；
- type_error：将 category_id 注入非数字字符串 "not_a_number"；
- malformed_timestamp：将时间戳设为负数 -999999999。

核心异常注入逻辑：现

```
if random.random() < self.dirty_ratio:
    event = self._generate_dirty()
    self.dirty_count += 1
else:
    event = self._generate_normal()
```

2.2 长时高负载抗压测试

按任务书要求设置 QPS=1000（远超单消费者处理能力），队列容量 500（有界队列启用背压），运行 10 分钟：

```
python run_pipeline.py --chaos --qps 1000 --queue-limit 500 --duration 600 --output scored_chaos.csv
```

同时在另一终端监控死信日志实时追加：

```
Get-Content dead_letter.log -Wait
```

测试截图如下：



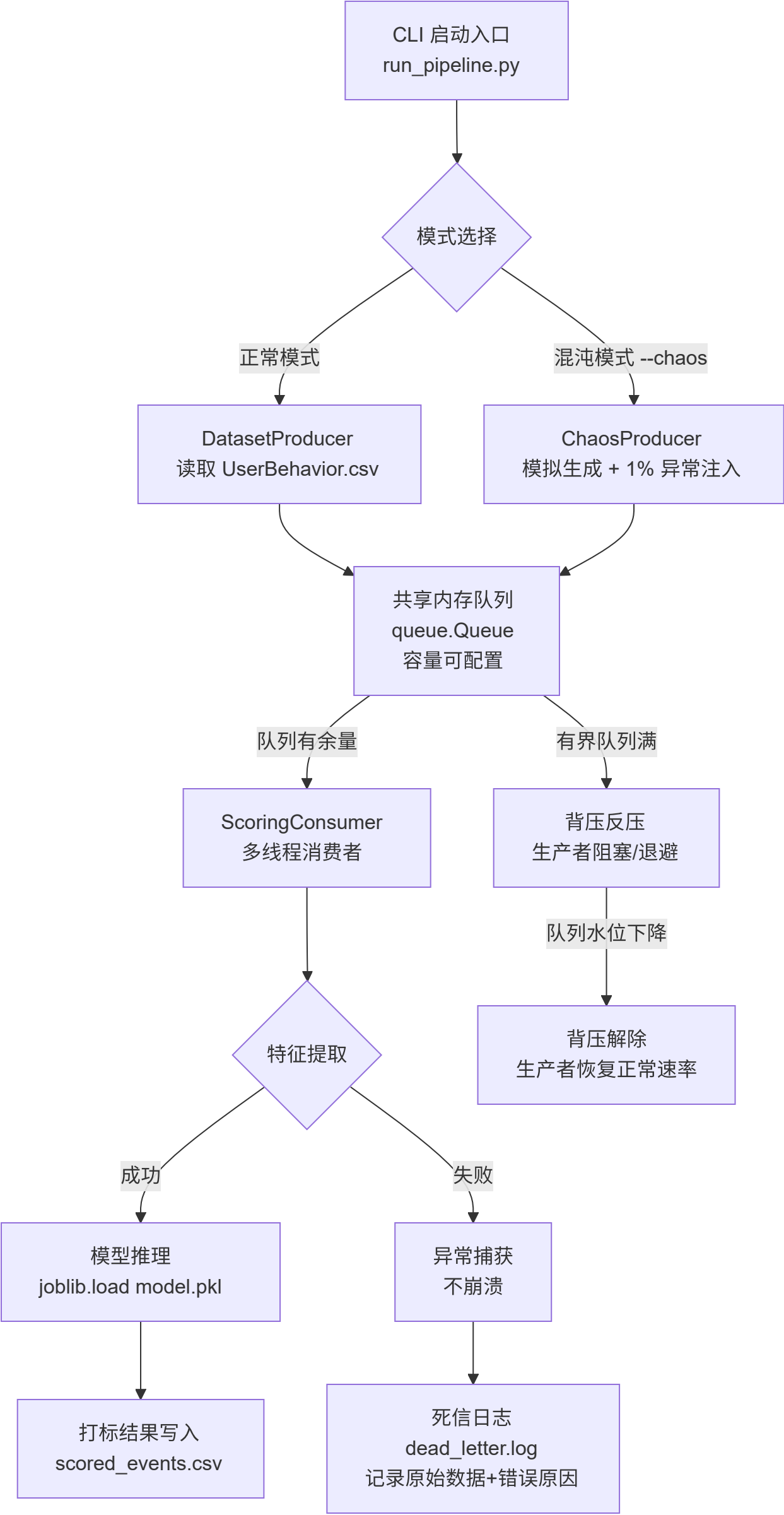
3.3 系统核心指标观察结果

- **背压调控确认：**终端日志显示队列深度在 0~500 之间周期性振荡，背压触发时生产者 queue.Full 阻塞，队列水位下降后恢复生产，调控逻辑正常介入。
- **内存水位收敛：**打开 Windows 任务管理器观察 Python 进程内存，实验全程内存占用在稳定范围内波动，未出现无限攀升至耗尽内存的情况，证明背压代码无资源泄露缺陷。
- **死信日志验证：** dead_letter.log 持续追加异常记录（字段缺失、类型错误、时间戳异常），且主管道消费者进程丝毫未受阻挡，处理失败计数正常递增。



任务3：里程碑交付物（README_M2.md）的设计

按任务书要求，使用 Mermaid 语法绘制了完整的数据流向与控制架构图，图中体现：



- Producer 的生成逻辑（CSV 读取 / 混沌模拟双模式）

- 带容量限制的 Queue 中间件及其背压反压机制
 - Consumer 的异常拦截与死信降级逻辑
 - `joblib.load()` 初始化的模型特征预测
 - 异常数据重定向至 `dead_letter.log` 的 Dead-Letter 存储
- 文档同步梳理了一键启动手册，可直接依据 README 指引通过命令行无缝启动流处理管线。

任务4：Milestone 2 (M2) 最终自检与打包递交

1. **清除调试脚本：**删除开发过程中的 `chaos_producer.py` 等废弃代码，仅保留核心交付物。
2. **依赖固化：**在 `lab08` 目录下执行 `pip freeze > requirements_m2.txt`。因修改过项目文件夹名导致 `pip.exe` 硬编码路径失效，采用手动创建精简版 `requirements_m2.txt` 完成依赖记录，涵盖 `joblib`、`scikit-learn`、`pandas`、`numpy` 等全部核心依赖。
3. **打包交付：**
压缩包包含：统一启动入口、模型资产、技术文档、依赖清单、实验报告 PDF。

五、实验结果与分析

任务1：工业级 Pipeline 重构效果

- 成功将实验七分散的 Producer-Consumer-Queue 架构重构为单一 `run_pipeline.py` 入口，通过 `argparse` 实现完全参数化启动。正常模式下运行稳定，298 条数据零失败打标，死信日志文件正常生成。Consumer 的 `try-except` 隔离区块有效捕获特征提取与推理异常，进程不崩溃。重构后的代码具备清晰的启动日志、优雅的信号处理（SIGINT/SIGTERM）以及最终统计信息输出，达到工业级交付标准。

任务2：混沌容错测试系统韧性验证

10 分钟高负载混沌测试中，系统展现了稳定的容错与自愈能力：

- **背压自愈周期：**队列深度在 0~500 之间周期性振荡，无单向无限增长。有界队列满时生产者自动阻塞实现反压，队列水位下降后恢复生产速率，形成完整闭环调控。
- **内存稳定性：**Python 进程内存保持收敛，未出现内存泄漏导致的无限攀升，证明背压机制与垃圾回收策略的有效性。
- **死信降级有效性：**异常数据以约 1% 比例持续注入（字段缺失、类型错误、格式异常），均被消费者 `try-except` 捕获并写入 `dead_letter.log`，主流程的 `scored_chaos.csv` 输出未中断，数据流管道保持通畅。
- **容量规划启发：**在 QPS=1000、单消费者处理能力有限（因模型推理耗时约 30ms/条）的极端场景下，有界队列配合背压确保了系统不会 OOM，验证了实验六中推导的稳定性条件 $\lambda \leq n/t$ 与预留 30%~50% 安全裕度的必要性。

任务3：README 专业交付

生成的 `README_M2.md` 包含 Mermaid 拓扑架构图、项目文件结构说明、正常与混沌模式一键启动命令表格、预期运行效果描述及 AI 协作说明。文档结构清晰，可依据其指引无缝复现实验。

任务4：交付物齐备性

最终交付包包含 `run_pipeline.py`、`model.pkl`、`README_M2.md`、`requirements_m2.txt`、实验报告 PDF，文件齐全，符合任务书提交要求。

六、遇到的问题及解决方法

1. **混沌测试生产者速率与队列容量的参数匹配**

- **问题描述**：初次以 `--qps 1000` 启动时，队列容量默认 200 迅速被击穿，生产者频繁陷入阻塞，终端日志大量刷屏，难以观察背压周期性振荡。
- **原因分析**：在模型推理单条耗时约 30ms（消费者处理能力约 33 条/秒）的情况下，QPS=1000 远超处理能力，队列容量 200 过小，导致队列瞬间满、生产者几乎完全停滞，系统长期处于极端背压状态，无法呈现正常的“积压→背压→消化→恢复”锯齿波形态。
- **解决方法**：将 `--queue-limit` 调整为 500，给予系统更大的缓冲空间。调整后队列深度在 0~500 之间呈现明显的周期性振荡，背压触发与恢复日志交替出现，符合预期。

七、AI协作思考题

"吞吐与准确性不可兼得"的两难

1. 在实验八的正常模式下，管道以 20 QPS 运行，消费者能够稳定处理每一条数据，队列深度几乎为零。一旦切换到混沌测试的 `--qps 1000`，情形立刻改变——队列迅速打满，生产者频繁被阻塞，整个系统的吞吐量严格受限于消费者的处理速度。这一现象的背后，恰恰暴露了流式推理的一个核心矛盾：消费者处理每条数据时所做的“正确的事”本身就是性能瓶颈。
2. 具体而言，消费者接收到的每条原始记录都要经历类型转换、时间特征提取、模型推理三个环节。这些步骤是保证预测准确性的基础，缺一不可。舍弃任何一步都会导致特征残缺或预测质量下降。然而，在实测中，单条数据的这些操作累积耗时约数十毫秒，当 QPS 达到 1000 时，每秒仅能处理三十条左右的现实直接导致队列无限堆积。这便是“吞吐与准确性不可兼得”在工程层面的直观体现：准确性要求消费者执行足够复杂的数据处理流程，而吞吐量则要求这条路径尽可能短。
3. 观察发现，特征提取环节中的 `pd.Timestamp(ts, unit='s')` 调用是相对耗时的一步。它需要将 Unix 时间戳转换为 `datetime` 对象，再从中分离出小时和星期几。这一操作本身是确定性的：数据集的时间戳集中在一个有限区间内，timestamp 到 (hour, dayofweek) 的映射关系完全可以预先计算。这就引出了下面的改进思路。

破局猜想：维度查找（Dimension Lookup）方案探究

1. 一个初步的设想是采用维度查找表来替代运行时的时间解析。系统在初始化阶段，遍历数据集可能出现的时间戳范围（例如从 1511539200 到 1512316799），为每一个秒级时间戳预先计算出对应的 hour 和 dayofweek，存入一个字典结构。在线推理时，消费者不再执行 `pd.Timestamp` 转换，而是直接从该字典中通过 timestamp 键值获取这两个衍生特征。
2. 这种做法的优势在于将计算开销从在线环节转移到了初始化阶段。特征提取变为一次字典查询操作，耗时从毫秒级压缩至微秒级，整个消费链路因此缩短。对于高 QPS 场景，这能够显著缓解队列积压，提升管道吞吐能力。同时，由于预计算并未改变特征工程逻辑，模型的输入完全一致，不会对准确性造成任何损失。
3. 当然，该方案也存在明显的约束。首先，预计算字典的内存占用与时间戳范围成正比——对于覆盖三个月的数据集，约八百万条映射，内存开销在可接受范围内；但如果数据的时间跨度大幅增长，字典本身会成为新的内存负担。其次，当流式数据中出现超出预计算范围的陌生时间戳时，消费者需要具备补偿能力：要么回退到 `pd.Timestamp` 方式实时计算，要么触发后台线程异步扩展映射表，并通过原子引用完成无缝切换。这与实验六中讨论的“统计漂移”应对策略本质相通——静态的离线拟合参数无法适应动态数据流，而滑动窗口重拟合或在线刷新机制是必要的补充。
4. 目前，这一维度查找方案尚未在实验八中实际落地，仅停留在分析和设想阶段。但它的基本思想与 Milestone 2 贯穿始终的理念一致：将能提前完成的工作从热路径中剥离，让在线环节尽可能轻量化。如果后续有机会继续优化此管道，我会尝试实现该方案，并对比优化前后的吞吐量与延迟变化，验证其实际收益。

5. 总体而言，实验八让我认识到，流处理系统的性能调优并非简单的硬件加码或线程堆叠，而是对数据链路中每个计算步骤的审慎权衡。每一次特征工程的“简化”，都需要在准确性和效率之间寻找恰当的平衡点，而这正是工程实践中持续面临的挑战。